

LogiChain API

Plateforme logistique événementielle

Module MP3 · Back-end & API REST

Clara Morin

La MANU (NOVEI) · Mai 2026



Le défi

12 000

personnes

500+

équipements

0

barre de réseau



Logistique complexe

festivals, salons, événements éco-responsables



Scans massifs

matériel scanné en masse sur le terrain



Réseau dégradé

boue, foule, sous-sol, connectivité instable



Impact temps réel

empreinte carbone mesurée à la volée



Pics de charge

montage et démontage concentrés sur quelques heures

Quatre objectifs



Centraliser et sécuriser

ressources matérielles et humaines sur une plateforme unique



Garantir l'offline-first

sur le terrain, sans perte de données quand le réseau lâche



Piloter par les KPI

stock, empreinte carbone, goulots mesurés en temps réel



Encaisser les pics d'utilisation

scans simultanés au montage et démontage sans dégrader le service

Ce que je présente aujourd'hui

- ▶ Back-end uniquement, l'API REST
- ▶ Repo séparé : logichain-api
- ▶ La PWA mobile-first viendra dans un autre Mini Projet
- ▶ Domaine prod : api-logichain.fulkia.fr

Architecture cible :



La stack

Couche	Choix
Runtime	Node.js 20 + TypeScript
Framework	Express
Base de données	MongoDB 7 (replica set rs0)
Object Data Model (ODM)	Mongoose
Validation	Zod (HTTP) + Mongoose (DB)
Auth	JWT + Bcrypt
Tests	Jest + ts-jest
Infrastructure	Docker multi-stage + Traefik HTTPS

Les contraintes imposées par le sujet



01

POO obligatoire

encapsulation, héritage justifié, polymorphisme



02

N-Tier strict

Entity / Repository / Service / Controller



03

REST Richardson niveau 2

verbes, URLs ressources, codes statut



04

Security by design

chiffrement, contrôle d'accès, validation



05

Robustesse offline

verrouillage optimiste pour la sync

ÉVALUATION

5 axes notés

Tout le code respecte ces cinq contraintes, démonstration dans les slides suivants.

✓ POO

✓ N-Tier

✓ REST L2

✓ Sécurité

✓ Offline

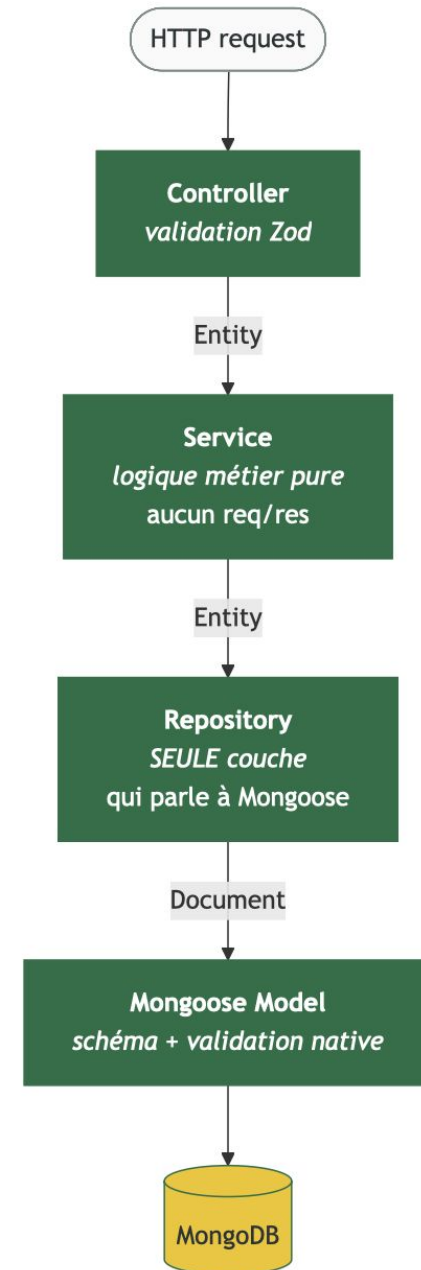
L'architecture N-Tier

Règle d'or

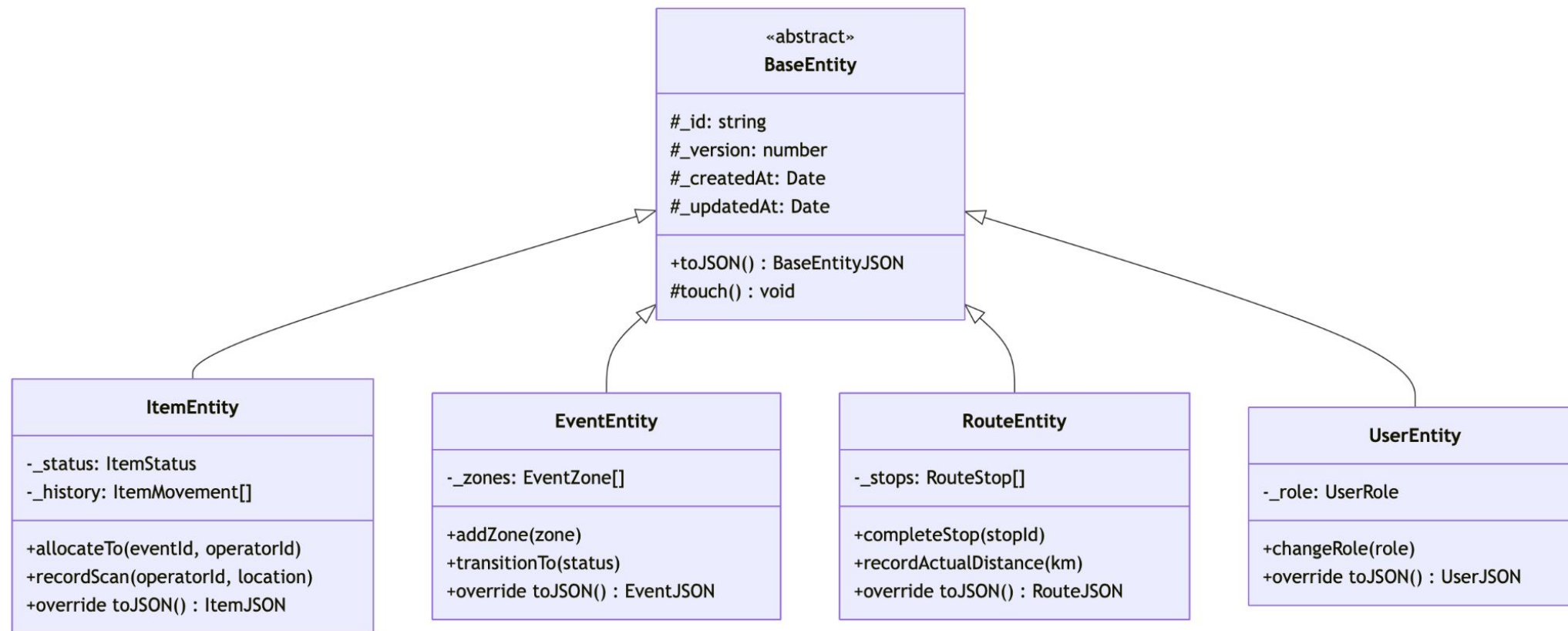
Chaque couche connaît uniquement la couche du dessous.

Le service ne sait pas qu'il y a du HTTP.

Le contrôleur ne sait pas qu'il y a une base.



Héritage justifié + Polymorphisme



- Héritage justifié : id/version/dates sont vraiment universels
- Polymorphisme : `toJSON()` surchargé par chaque entité (`super.toJSON()` + champs propres)
- Le contrôleur appelle `entity.toJSON()` sans connaître la classe concrète

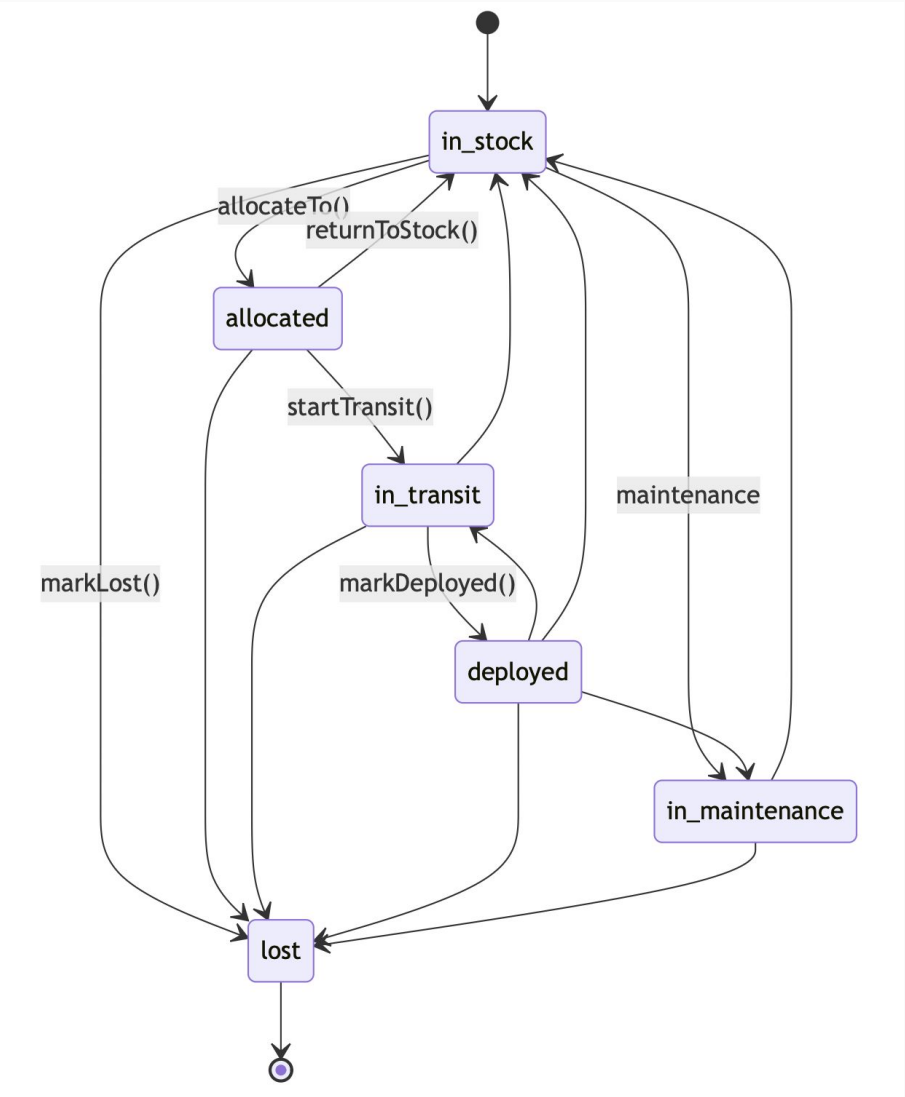
Encapsulation : l'entité protège son état



```
export class ItemEntity extends BaseEntity {  
  private _status: ItemStatus;  
  private _history: ItemMovement[];  
  
  public get status(): ItemStatus { return this._status; }  
  
  public allocateTo(eventId: string, operatorId: string): void {  
    this.assertTransition('allocated'); // ← machine à états  
    this._status = 'allocated';  
    this.appendMovement({ ... });  
    this.touch();  
  }  
}
```

Aucun setter public · transitions contrôlées par une machine à états

Cycle de vie d'un item (machine à états)



Etat	Signification
in_stock	Dans l'entrepôt, disponible
allocated	Réservé à un événement précis
in_transit	En cours de transport
deployed	Installé sur site
in_maintenance	En réparation
lost	Perdu (état terminal)

Comment ai-je modélisé mes données dans MongoDB, et pourquoi ?

Collection	Particularités clés
items	QR unique · GeoJSON Point + 2dsphere · historique de mouvements imbriqué
events	GeoJSON Polygon par zone · slug unique · cycle de vie
routes	Stops imbriqués · mode transport · distance réelle / planifiée
users	Bcrypt en select:false · rôles enum · email unique
notifications	alertes temps réel (SSE + Change Streams) · + collection Time Series monitoring_metrics

- **Dénormalisation assumée**
L'historique des mouvements est imbriqué dans le document item -> On le lit toujours avec l'item, un seul accès disque, zéro jointure. Trade-off assumé : lecture ultra rapide.
- **Validation côté Mongoose (pas natif MongoDB)**
Schémas TypeScript stricts côté ORM : autocomplétion, types portables, indépendance du moteur.
- **Index composés ciblés**
({eventId, status} = un seul index qui sert les requêtes filtrant sur les deux champs), index unique (qrCode pour le scan), index géospatial 2dsphere (pour les requêtes \$near).
- **optimisticConcurrency: true**
Mongoose incrémente __v automatiquement à chaque save. C'est la fondation du verrouillage optimiste.

Verrouillage optimiste, le filet pour la sync offline

Pessimiste, non retenu

Verrouiller avant d'écrire.

On pose un lock, on modifie, on libère. Sûr pour la cohérence mais bloquant, impossible à demander à des agents terrain qui scannent hors-ligne.

```
await this.model.findOneAndUpdate(
  { _id: id, __v: expectedVersion }, // ← matche sur la version
  { ...update, $inc: { __v: 1 } },
  { new: true }
);
```

- ▶ Mongoose maintient __v automatiquement à chaque save
- ▶ Chaque update matche sur { _id, __v }, opération atomique
- ▶ Si findOneAndUpdate retourne null → ConflictError 409
- ▶ La PWA reçoit 409 et déclenche un merge côté client

Optimiste, mon choix

Lire, modifier, vérifier la version au commit.

Compare-and-swap atomique sur __v. Pas de blocage. Les conflits sont détectés au moment de l'écriture, exactement ce qu'il faut pour le offline.

Scénario terrain : pourquoi c'est critique

09h00

Agent A scanne hors-ligne dans un sous-sol.

09h05

Agent B scanne le même item en ligne. __v passe à 2.

09h12

Agent A retrouve le réseau et POST avec __v: 1.

→ **409**

Le serveur détecte le conflit. La PWA re-GET, merge côté client, re-soumet.

Transactions ACID et allocation atomique d'un lot

Réserver d'un coup tout le matériel d'un secteur, sans jamais laisser un état incohérent

LE PROBLÈME

Au montage, on réserve d'un coup tout le matériel d'un secteur (50 items). Si l'item n°37 est déjà perdu, que deviennent les 36 déjà traités ? Sans transaction → **état incohérent, ingérable en offline.**

LA SOLUTION

repo.withTransaction() ouvre une session sur le replica set **rs0**. Tout-ou-rien : un seul échec (404 / 422 / 409) annule l'intégralité du lot. Le verrouillage optimiste **__v** reste actif → isolation.

```
POST /api/v1/events/:id/allocate { itemIds: [...] }
```

A

Atomicité

rollback auto si un item échoue

C

Cohérence

chaque item validé par sa machine à états

I

Isolation

__v + snapshot transactionnel

D

Durabilité

commit répliqué sur rs0

Conventions REST, Richardson niveau 2

Modèle de maturité de Richardson

0	RPC sur HTTP
1	Ressources
2	Verbes + statuts
3	HATEOAS sur-engineering pour notre PWA

Action	Verbe	URL
Lister	GET	/api/v1/events/:id/items?status=in_stock
Créer	POST	/api/v1/items
Modifier	PATCH	/api/v1/items/:id
Supprimer	DELETE	/api/v1/items/:id
Action métier	POST	/api/v1/items/:id/scan · /allocate · /deploy

Codes statut clés, trois échecs, trois codes sémantiquement distincts

200 OK	201 Created	204 No Content	400 Bad Request Input mal formé (Zod refuse)
401 Unauthorized	404 Not Found	403 Forbidden Rôle insuffisant (RBAC requireRole)	409 Conflict Version OCC ou clé unique
			422 Unprocessable Règle métier refuse

Security by design

Contrôle d'accès strict et validation systématique du token jusqu'à la base



Authentication

JWT Bearer (HS256), secret $\geq$ 32 caractères validé au boot (Zod fail-fast)



Mots de passe

Bcrypt 12 rounds · passwordHash en select:false → jamais sérialisé



Autorisation

RBAC 4 rôles via requireRole() → 403 Forbidden (admin · logistics_manager · field_agent · transporter)



Validation en profondeur

Zod (entrée HTTP) puis Mongoose (avant écriture en base)



Durcissement HTTP

Helmet · CORS allow-list · body limit 1 Mo · x-powered-by off



Transport

HTTPS Let's Encrypt (Traefik) · MongoDB jamais exposé publiquement

KPI temps réel par agrégation

Un seul pipeline MongoDB, plusieurs métriques en un passage

- ▶ **Pipeline \$facet** : aggregateByEvent : stocks par statut et par catégorie, poids total, CO₂ de fabrication cumulé : en une seule requête
- ▶ **Isolé dans le Repository** : les pipelines d'agrégation sont confinés à la couche d'accès aux données (exigence du sujet)
- ▶ **Alimente le pilotage** : tableau de bord, empreinte carbone consolidée, détection des goulots d'étranglement
- ▶ **Endpoints** : /dashboard/events/:id/carbon-footprint · /dashboard/allocations/check

```
{ $match: { eventId } },  
{ $facet: {  
  byStatus: [ { $group: { _id: '$status', count: { $sum: 1 } } } ],  
  byCategory: [ { $group: { _id: '$category', count: { $sum: 1 } } } ],  
  sums: [ { $group: { _id: null, weight: { $sum: '$weightKg' } } } ]  
} }
```


Empreinte carbone et méthodologie ADEME



Fabrication amortie

$$\text{co2} = (\text{fabricationCo2} / \text{dureeVie}) \times (\text{joursEvenement} / 365)$$



Transport (Base Carbone ADEME)

$$\text{co2} = \text{tonnes} \times \text{distanceKm} \times \text{facteurMode}$$

Mode de transport	kgCO2e / tonne•km	Source
Camion thermique 7,5-26t	0,058	API ADEME live
Camion électrique	0,020	Fallback
Rail (fret ferroviaire)	0,00401	API ADEME live
Vélo cargo	0	Fallback



DEUX COMPOSANTES

Fabrication amortie

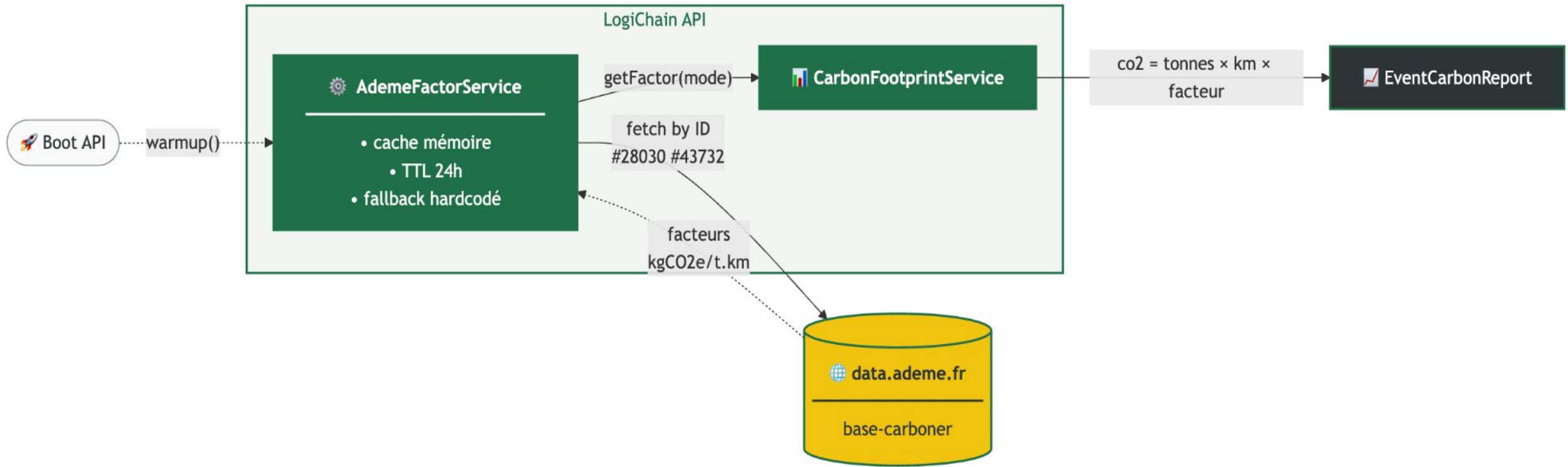
Un projecteur qui sert 8 ans sur 50 événements ne peut pas compter à 100 % sur chacun. On divise par la durée de vie, puis on multiplie par le ratio jours-événement / 365.

Transport ADEME

Tonnes × distance × facteur. Les facteurs proviennent de l'API ADEME en live ; en cas d'échec on retombe sur les valeurs Base Carbone embarquées.

Précision : au centime de kgCO₂e

Intégration API Open Data ADEME



- Endpoint ADEME : `data.ademe.fr/.../base-carboner`
- Sélection par identifiant ADEME (#28030, #43732) → robuste

- Pattern stale-while-revalidate
- Fallback hardcodé si ADEME down → service jamais HS
- Endpoint debug : `GET /dashboard/emission-factors`

Allocation sans goulot, algorithme du banquier (Dijkstra 1965)

Mapping OS → LogiChain : on prouve la sûreté avant d'accorder une ressource

OS classique		LogiChain
processus	→	événement
ressource	→	catégorie d'item
max claim	→	besoin prévisionnel
allocation	→	items alloués
available	→	items in_stock



État sûr

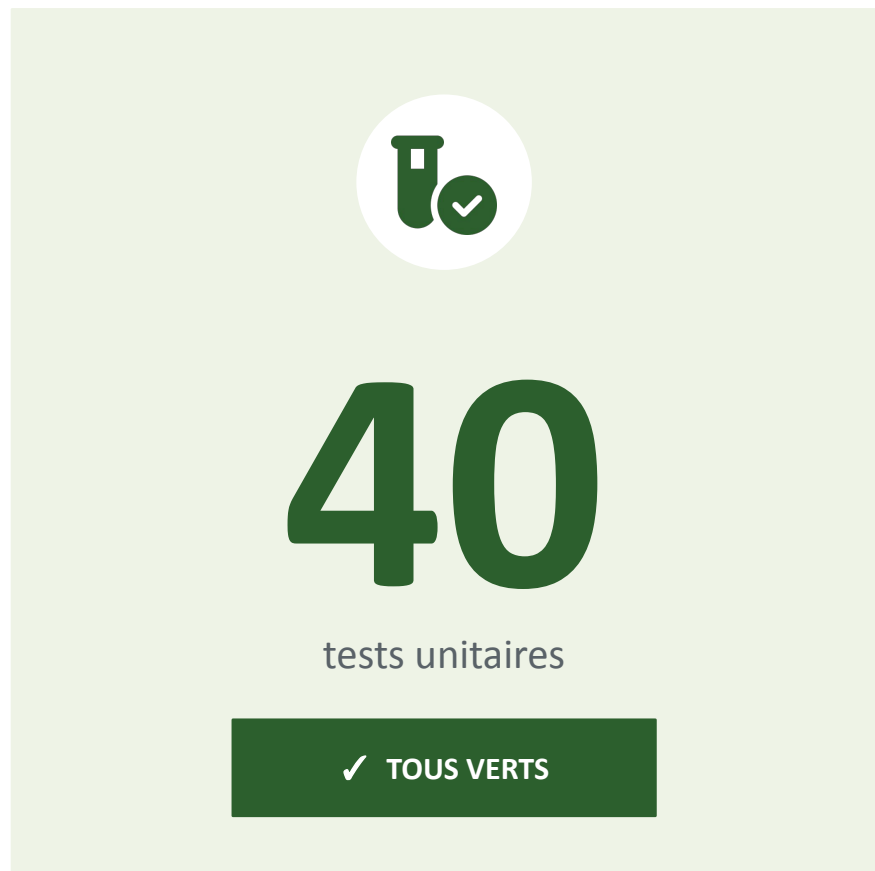
DÉFINITION

Il existe une séquence d'événements où chaque besoin peut être satisfait à la suite, en récupérant les ressources libérées par les événements déjà servis.

- ✓ **Avant chaque allocation**, on simule, puis on vérifie la sûreté
- ✓ **Si l'état devient non-sûr**, refus HTTP 422 (Unprocessable Entity)
- ✓ **Empêche les goulots**, d'étranglement structurels avant qu'ils n'arrivent

Tests et qualité

Couverture des points sensibles + 1 test de charge k6 (pics de scan)



- ✓ Focus services à logique riche (exigence sujet)
- ✓ Repositories mockés (pas de DB en unitaire)
- ✓ Formule ADEME vérifiée au centime
- ✓ Exemple historique Dijkstra à 5 processus
- ✓ Transitions sûr ↔ non-sûr couvertes
- ✓ Immutabilité de ResourceState garantie
- ✓ TypeScript strict (noUncheckedIndexedAccess)
- ✓ Zero any non justifié
- ✓ Intégration live API ADEME
- ✓ Allocation transactionnelle ACID : atomicité et rollback couverts

Documentation interactive de l'API

OpenAPI 3 généré depuis les schémas Zod, servi par Scalar sur /docs

- ✓ **Single source of truth**
le même schéma Zod valide la requête ET génère la doc OpenAPI → jamais désynchronisée
- ✓ **UI Scalar interactive**
43 endpoints documentés · « try it out » · bouton Authorize (Bearer JWT)
- ✓ **Protégée par Basic Auth**
/docs et /openapi.json derrière login · comparaison à temps constant (timingSafeEqual)
- ✓ **La vitrine de l'API**
sans frontend

GET <https://api-logichain.fulikia.fr/docs>

The screenshot shows the Scalar API documentation interface for the LogiChain API. The browser address bar displays `api-logichain.fulikia.fr/docs#description/introduction`. The interface includes a search bar, a sidebar with a table of contents, and a main content area.

Table of Contents (Sidebar):

Category	Endpoint	Method
Auth	Authentification (JWT)	POST
	Renouveler le token	POST
	Créer un utilisateur (admin)	POST
	Profil courant	GET
	Items	
Items	Lister les items	GET
	Créer un item	POST
	Item par code QR (scan)	GET
	Item par id	GET
	Modifier un item	PATCH
	Supprimer un item	DEL
	Scan terrain (géolocalisé)	POST
	Allouer à un événement	POST
	Démarrer le transit	POST
	Déployer sur site	POST
	Déclarer une anomalie	POST
	Marquer perdu	POST
	Retour au stock	POST
Events		

Main Content Area:

- LogiChain API**
v1.0.0 OpenAPI 3.0.0
Download OpenAPI Document
- API REST de la plateforme logistique événementielle LogiChain (MP3). Documentation générée automatiquement depuis les schémas de validation Zod. Authentification : POST /api/v1/auth/Login puis bouton **Authorize** (Bearer JWT).
- Auth**
Authentification & utilisateurs
- Operations**
 - POST /api/v1/auth/login
 - POST /api/v1/auth/refresh
 - POST /api/v1/auth/register
 - GET /api/v1/auth/me
- Server**
https://api-logichain.fulikia.fr
Production
- Authentication**
Auth Type: No authentication selected
- Client Libraries**
Shell, Ruby, Node.js, PHP, Python, More

Notifications temps réel

Change Streams MongoDB → push SSE aux agents, sans polling

- ✓ **Détection par Change Streams**
MongoDB surveille les écritures en base ; le serveur réagit à la source, aucun polling
- ✓ **Push via Server-Sent Events**
l'agent connecté reçoit l'alerte en direct : anomalie, perte d'item, annulation
- ✓ **Persistées pour l'offline**
collection « notifications » : l'agent reconnecté récupère les alertes manquées
- ✓ **~7 ms bout en bout**
du POST /anomaly à la réception SSE — mesuré en démo live

GET /api/v1/notifications/stream

flux SSE

4ms	All
> Cookies	
> Request Headers 4	
> Response Headers 20	
▼ Body	Listening... Cancel
event: connected	
data: {"ok":true}	
event: notification	
data: {"id":"6a452d0fbd381a11297f79b0","version":0,"createdAt":"2026-07-01T15:06:55.164Z","updatedAt":"2026-07-01T15:06:55.164Z","type":"anomaly","severity":"critical","title":"Anomalie — Découpe asservie 300W","message":"Test live","eventId":"6a451beffe237b06b3f8de21","itemId":"6a451beffe237b06b3f8de30","audience":["admin","logistics_manager","field_agent"],"read":false}	
: ping	
: ping	

Déploiement et préparation pour la suite



EN LIGNE

Déployé en production

- ✓ Dockerfile multi-stage Alpine non-root
- ✓ docker-compose : Mongo rs0 auto-init + API
- ✓ Traefik HTTPS auto avec Let's Encrypt
- ✓ Domaine api-logichain.fulkia.fr prêt
- ✓ Mongo non exposé publiquement



À VENIR

Préparé pour le front + MP4/5

- Verrouillage optimiste actif → sync offline
- Format JSON stable + pagination
- Endpoint /items/by-qr/:code (scan mobile)
- Temps réel SSE + Change Streams actif → WebSocket front
- Monitoring Time Series actif → dashboards Grafana

En ligne sur <https://api-logichain.fulkia.fr> : la PWA se branchera sans modification.

Récapitulatif

- ✓ N-Tier strict, POO démontrée à tous les niveaux
- ✓ REST Richardson 2, security by design
- ✓ MongoDB optimisé : index, GeoJSON, OCC
- ✓ Deux services métier signature + 40 tests
- ✓ Infrastructure Docker déployé
- ✓ Temps réel : SSE + Change Streams (~7 ms)

LogiChain API

Plateforme logistique événementielle

Module MP3 · Back-end & API REST

Merci pour votre attention

Des questions ?

Clara Morin

La MANU (NOVEI) · Mai 2026